

**SIMATS ENGINEERING
ASSESSMENT TOOL**

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Application Based Question

Code of Conduct:

I U.Divya/192372277 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary

action. Signature of the student: Divya

**SIMATS ENGINEERING
ASSESSMENT TOOL**

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet $\Sigma = \{0,1\}$ to validate passwords that **end with the pattern "01"**. Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

Problem Statement

Design a Deterministic Finite Automaton (DFA) over the alphabet

$\Sigma=\{0,1\}$

that accepts all strings ending with **"01"**. Minimize the DFA and explain how minimization improves hardware efficiency.

State Design

States

- **q0** – Initial state
- **q1** – Last symbol read is **0**
- **q2** – String ends with **01** (Accept State)

Transition Table

Current State	Input 0	Input 1
->q0	q1	q0
q1	q1	q2
*q2	q1	q0

(*Accept State)

Explanation

- The DFA begins at **q0**.

- Reading **0** moves to q1.
- Reading **1** after q1 reaches q2.
- q2 accepts only when the last two symbols are **01**.
- Any additional input updates the state according to the suffix of the string.

Examples

01-Accepted

1101-Accepted

0001-Accepted

1010-Rejected

111-Rejected

Minimized DFA

The DFA already has only **three distinguishable states**.

Therefore,

Number of states before minimization = 3

Number of states after minimization = 3

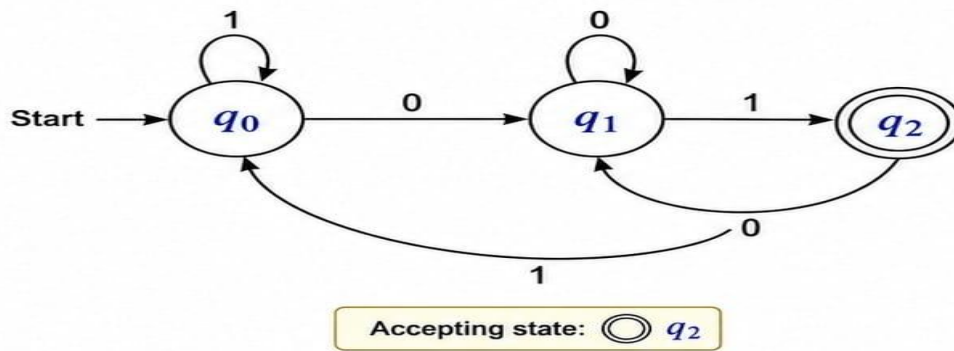
No further reduction is possible.

Hardware Efficiency

The minimized DFA improves embedded systems by

- reducing memory usage
- reducing hardware logic gates
- lowering power consumption
- providing faster password verification
- improving reliability in ATM machines and authentication devices

1. DFA for strings ending with "01" over $\Sigma = \{0,1\}$



2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

Regular Expression

aaaabb

or equivalently

$a\{4\}b\{2\}$

Rejected Strings

aaabb

aaaaab

aaaabbb

abb

Working

The expression requires

- exactly four a's
- followed by exactly two b's

No additional symbols are allowed.

Applications in Web Input Validation

Regular expressions are widely used for validating

- Email format
- Password rules
- Phone numbers
- PIN numbers
- Usernames
- Postal codes
- Date formats

Examples

Email

[^\[A-Za-z0-9._%+-\]+@\[A-Za-z0-9.-\]+\.\[A-Za-z\]{2,}\\$](#)

Example

[abc@gmail.com](#)

Password

[^\(?=.*\[A-Z\]\)\(?=.*\[a-z\]\)\(?=.*\d\).\{8,\}\\$](#)

Ensures

- one uppercase
- one lowercase
- one digit
- minimum 8 characters

Phone Number

[^\[0-9\]{10}\\$](#)

PIN Code

$^{[0-9]\{6\}}\$$

Advantages

- Fast validation
- Reduces invalid inputs
- Prevents malformed data
- Improves application security
- Reduces server workload

2. Transition Table for DFA (strings ending with "01")

State	Input 0	Input 1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

Start state: q_0 Accepting state: q_2

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a^*b^* can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

Regular Expression

a^*b^*

Meaning

- Zero or more a's
- followed by
- Zero or more b's

Examples Accepted

ϵ

a

aa

aaa

ab

aab

aaabbb

bbb

Rejected

ba

abab

baba

abba

Finite Automaton

States

- q_0 (Start)
- q_1 (Reading b's)
- q_d (Dead State)

Transition Table

Current State	Input 0	Input 1
$\rightarrow q_0$	q_0	q_1
q_1	q_d	q_1
q_d	q_d	q_d

Why Regular Expressions and Finite Automata are Equivalent

Every regular expression can be converted into

- DFA
- NFA

Likewise,

every DFA/NFA can be converted back into an equivalent regular expression.

Therefore,

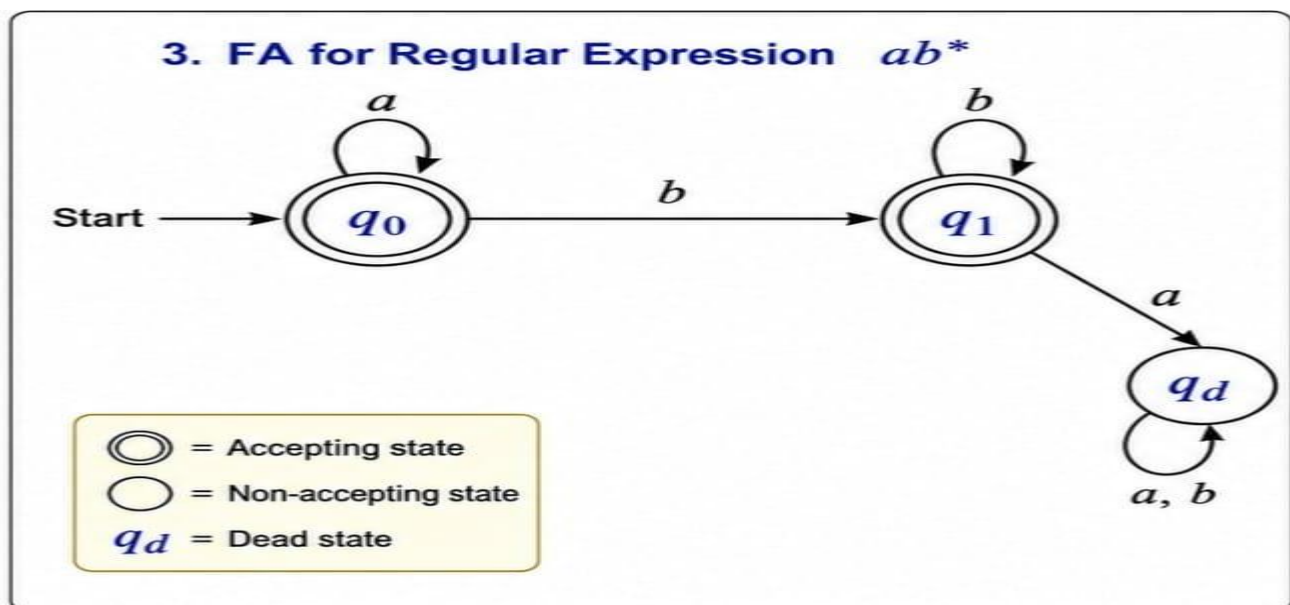
Regular Expression $\Leftrightarrow$ Finite Automaton

Both recognize exactly the **regular languages**.

Applications

Used in

- Intrusion Detection Systems
- Network packet filtering
- Malware detection
- Lexical analysis
- Log monitoring
- Spam filtering



4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L = \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

Language

$$L = \{ a^n b^n \mid n \geq 0 \}$$

Pumping Lemma

If L is regular,

there exists a pumping length p .

Choose

$$w = a^p b^p$$

According to the lemma,

$$w = xyz$$

such that

$$|xy| \leq p$$

$$|y| > 0$$

Since

$$|xy| \leq p$$

both x and y contain only a 's.

Assume

$$y = a^k$$

Where

$$k > 0$$

Pump down

xy^0z

Result

$a^{(p-k)}b^p$

Now

Number of a's $\neq$ Number of b's

Hence

$a^{(p-k)}b^p \notin L$

This contradicts the Pumping Lemma.

Therefore,

L is NOT Regular.

Implications in Compiler Design

Finite automata cannot recognize

- balanced brackets
- nested loops
- recursive function calls
- matching begin-end blocks

Compilers therefore use

- Context-Free Grammars (CFG)
- Pushdown Automata (PDA)
- Parsing algorithms

Examples

if{

while{

}

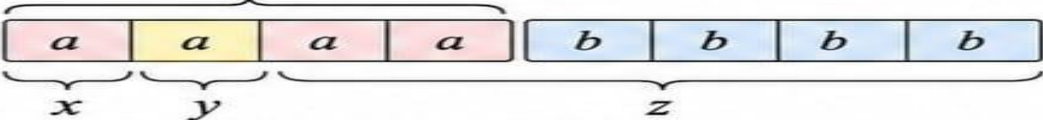
}

The nesting requires a stack, which DFAs do not have.

4. Pumping Lemma for $L = \{a^n b^n \mid n \geq 0\}$

Choose string $w = a^4 b^4$ ($p \geq 4$)

$|xy| \leq p$

$w =$ 

Pump down: $i = 0$ (remove y)

$xy^0z =$ 

✗ Not in L

Number of a 's $\neq$ Number of b 's $\Rightarrow$ Not in L
 Contradiction to Pumping Lemma $\Rightarrow$ **L is not regular.**

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

Regular languages remain regular after applying operations such as

- Union ($\cup$)
- Intersection ($\cap$)
- Complement ($'$)
- Difference
- Concatenation
- Kleene Star

Example

Suppose

$R1$ = strings containing "virus"

$R2$ = strings ending with ".exe"

Union

$R1 \cup R2$

Detects

- virus strings
- executable files

Intersection

$R1 \cap R2$

Detects executable files containing "virus".

Complement

R'

Detects all strings **not** matching the rule.

Concatenation

admin + login

Matches

Adminlogin

Kleene Star

$(ab)^*$

Matches

ϵ

ab

abab

ababab

Why the Result Remains Regular

Closure properties guarantee that applying these operations to regular languages always produces another regular language. Since regular languages can be recognized by finite

automata, the combined detection rules can still be implemented efficiently using DFAs or NFAs.

Applications

- Intrusion Detection Systems
- Spam filtering
- Email filtering
- Network firewalls
- Input validation
- Log analysis
- Compiler lexical analysis

Conclusion

Regular expressions and finite automata provide efficient mechanisms for recognizing patterns used in password validation, web input validation, intrusion detection, and compiler lexical analysis. DFA minimization reduces hardware complexity, closure properties allow multiple detection rules to be combined while preserving regularity, and the Pumping Lemma demonstrates the limitations of finite automata, motivating the use of context-free grammars and pushdown automata for more complex language recognition.

SIMATS ENGINEERING ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2– 3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SIMATS ENGINEERING ASSESSMENT TOOL

2. Can enforce rules like:

- **Email format**
- **Password rules**
- **Input structure**